

Extra Output (to click)→Override
Default (to click)→Type Project Name.
hex→Output Format→intel-standard
(to select)

Press 'ok' after all the above settings are done.

The source code for LED blinking (main.c) along with comments for instructions (also shown in the text editor window in Fig. 3) is:

```
//A Sample code for switching a port
pin PA0 high for 0.5 seconds and low
//for 0.5 seconds
#include <iavr.h> //This
includes appropriate header file for the
//device as selected
#include <intrinsics.h> //Library
header file to use __delay_cycles()

//Macro definition for reversing a bit
#define REVERSEBIT(ADDRESS, BIT)
(ADDRESS ^= (1 << BIT))

//Assuming Default Internal clock of
16.0 Mhz
__C_task void main(void) //Main
function
```

```

{
    DDRA = 0x01; //Set Data
    Direction Register for PA0 as output
    while(1) //An
    infinite looping begins
    {
        REVERSEBIT(PORTA,
        PA0);
        __delay_cycles(500000);
    }
    //Reverse bit after 0.5 seconds
}

```

After entering the code, it is ready for compilation and linking to generate the .d90 debug file and Intel standard hex file for downloading into the target microcontroller. These files are generated in Project→Debug→Exe folder. To compile the code, click 'Rebuild All' option in 'Project' menu.

Save the embedded workbench project (.ewp) file to the working folder. After this, you are done with coding and IAR Embedded Workbench.

AVR Studio. After compilation, start simulation and debugging using AVR Studio 4.0 from 'All Programs'

menu of Windows. When the application starts, open the .d90 debugging file located in Project\Debug\Exe folder and save the AVR Studio project (.aps) file to 'Project' folder. Now from the selection window, select the platform and device for simulation as AVR Simulator and ATmega8515, respectively. This will start simulating the generated C code in IAR Embedded Workbench. You may continuously edit and recompile the code in parallel with AVR Studio, which automatically reloads the object file and C file for simulation (as shown in Fig. 4).

Immediately before doing anything else, change the system clock frequency to correct value so that the code runs with correct clock frequency. Now proceed as follows:

Main menu→Debug→AVR Simulator Options (Alt+O)

You can keep pressing Debug→Step Into (F11) to proceed with simulation. Expand the port which is to be monitored (Port A for the current code). As soon as PA0 is set right-

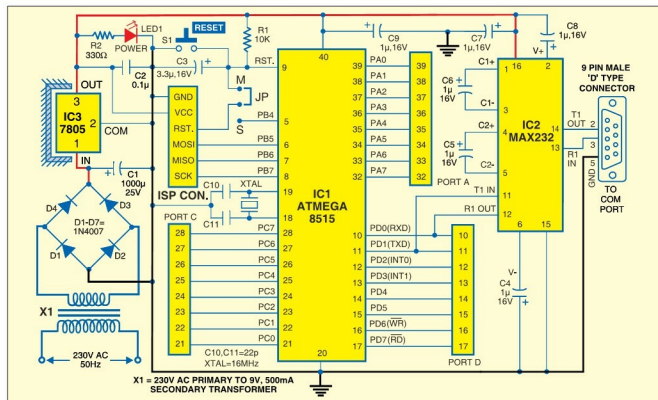


Fig. 5: Circuit of the development board for ATmega8515